

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN- ỨNG DỤNG



BÀI BÁO CÁO

MÔN: KHAI PHÁ DỮ LIỆU

Đề tài: **THỰC HÀNH THUẬT TOÁN PHÂN LỚP**

Giảng viên hướng dẫn : Đỗ Như Tài

Sinh viên thực hiện : Họ và tên

Mã số sinh viên

Phạm Ngọc Phương Uyên

3123580061

Trần Thu Hiền

3123580013

Nguyễn Châu Nhật Tường

3123580060

Lớp : DDU1231

Thành phố Hồ Chí Minh, tháng 10 năm 2025

LỜI CẢM ƠN

Trước tiên, nhóm sinh viên chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Đỗ Như Tài – giảng viên phụ trách môn Khai phá dữ liệu, thầy đã tận tình giảng dạy, hướng dẫn và hỗ trợ chúng em xuyên suốt quá trình học tập cũng như trong quá trình thực hiện bài báo cáo này. Những kiến thức, định hướng và góp ý từ thầy là nền tảng quan trọng giúp nhóm chúng em hoàn thiện bài báo cáo một cách hiệu quả và thực tế hơn.

Chúng em cũng xin trân trọng cảm ơn toàn thể quý thầy cô Khoa Toán – Ứng dụng, đã tạo điều kiện thuận lợi về môi trường học tập, tài liệu và cơ sở vật chất để chúng em có thể phát huy khả năng nghiên cứu, rèn luyện tư duy và kỹ năng làm việc nhóm.

Bên cạnh đó, nhóm cũng xin ghi nhận và cảm ơn sự hỗ trợ, phối hợp chặt chẽ giữa các thành viên trong nhóm trong suốt thời gian làm việc. Mỗi thành viên đều có sự đóng góp tích cực về nội dung, kỹ thuật và hình thức trình bày, góp phần tạo nên một bài đồ án hoàn chỉnh như hiện tại.

Mặc dù đã nỗ lực rất nhiều trong quá trình thực hiện, nhưng với kiến thức và kinh nghiệm còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét quý báu từ thầy và các bạn để có thể tiếp tục hoàn thiện trong các đề tài, báo cáo sau này.

Một lần nữa, chúng em xin chân thành cảm ơn!

BẢNG PHÂN CHIA CÔNG VIỆC

Thứ tự	Họ và tên	Công việc phụ trách	Mức độ hoàn thành
1	Nguyễn Châu Nhật Tường	- Phụ trách nội dung xây dựng mô hình Naive Bayes - Hoàn thiện bài báo cáo	100%
2	Phạm Ngọc Phương Uyên	- Phụ trách nội dung mô hình Support Vector	100%
3	Trần Thu Hiền	-Phụ trách nội dung xây dựng mô hình cây quyết định và rừng cây	100%

Mục Lục

PHẦN I: CÂY QUYẾT ĐỊNH VÀ RỪNG CÂY	5
1.1 ÔN TẬP LÝ THUYẾT	5
1.2. BÀI TẬP	10
<i>Bài tập thực hành 1: Xây dựng cây quyết định và rừng cây trên dữ liệu Titanic</i>	10
<i>Bài tập thực hành 2: Xây dựng cây quyết định và rừng cây trên dữ liệu bệnh tiểu đường.</i>	21
PHẦN II: SUPPORT VECTOR MACHINE (SVM)	32
2.1. ÔN TẬP LÝ THUYẾT	32
2.2. BÀI TẬP	34
<i>Bài tập thực hành 1: Xây dựng mô hình từ giải thuật SVM trên dữ liệu bệnh tiểu đường.</i>	34
<i>Bài tập thực hành 2: Xây dựng mô hình từ giải thuật SVM trên dữ liệu các con thú trong rừng.</i>	39
PHẦN III: BAYES NGÂY THƠ (NAIVE BAYES)	45
1.1. ÔN TẬP LÝ THUYẾT	45
1.2. BÀI TẬP	48
<i>Bài tập thực hành 1: Xây dựng mô hình Naïve ngây thơ trên tập dữ liệu hành vi của khách hàng.</i>	48
<i>Bài tập thực hành 2: Xây dựng mô hình Naïve ngây thơ trên tập dữ liệu mushroom.</i>	50

PHẦN I: CÂY QUYẾT ĐỊNH VÀ RỪNG CÂY

1.1 ÔN TẬP LÝ THUYẾT

1. Quy trình khai phá dữ liệu CRISP-DM là gì? Quy trình khai phá dữ liệu SEMMA là gì?

CRISP-DM (Cross Industry Standard Process for Data Mining) Là quy trình chuẩn công nghiệp, tập trung vào mục tiêu kinh doanh gồm 6 giai đoạn:

1. **Business Understanding:** Hiểu mục tiêu kinh doanh, vấn đề cần giải quyết.
2. **Data Understanding:** Thu thập dữ liệu ban đầu, mô tả, khám phá và đánh giá chất lượng dữ liệu.
3. **Data Preparation:** Làm sạch, tổng hợp, chọn lựa biến, tiền xử lý dữ liệu.
4. **Modeling:** Chọn thuật toán mô hình, huấn luyện, điều chỉnh tham số.
5. **Evaluation:** Đánh giá mức độ đáp ứng và yêu cầu kinh doanh.
6. **Deployment:** Triển khai mô hình vào thực tế.

SEMMA (Sample, Explore, Modify, Model, Assess):

1. **Sample:** Lấy mẫu dữ liệu đại diện.
2. **Explore:** Khám phá dữ liệu, tìm quan hệ, phát hiện bất thường, xu hướng.
3. **Modify:** Xử lý dữ liệu, chọn đặc trưng, biến đổi dữ liệu.
4. **Model:** Xây dựng mô hình dự đoán.
5. **Assess:** Đánh giá mô hình.

2. Cây quyết định hoạt động như thế nào? Hãy giải thích các thành phần chính (nút gốc, nút lá, nhánh) và cách cây đưa ra dự đoán.

Thành phần của cây quyết định:

- **Nút gốc (root):** Nút trên cùng của cây, đại diện cho toàn bộ tập dữ liệu, nơi quá trình phân chia bắt đầu.
- **Nhánh (Branches):** Các đường nối giữa các nút, đại diện cho kết quả của một phép kiểm tra/điều kiện trên một đặc trưng cụ thể.

- **Nút nội bộ (Internal Nodes):** Các nút nằm giữa nút gốc và nút lá, nơi diễn ra các quyết định phân chia tiếp theo.
- **Nút lá (Leaf Nodes):** Các nút cuối cùng không thể phân chia nữa. Chúng chứa kết quả dự đoán cuối cùng (nhãn lớp hoặc giá trị số).

Cách hoạt động:

Cây quyết định chia nhỏ tập dữ liệu thành các tập con nhỏ hơn dựa trên các quy tắc điều kiện (If-Then) liên quan đến các đặc trưng (features). Quá trình này lặp lại cho đến khi đạt được điều kiện dừng (ví dụ: nút thuần khiết hoặc đạt độ sâu tối đa).

Cách dự đoán:

Dữ liệu đầu vào đi từ nút gốc, kiểm tra các điều kiện tại từng nút nội bộ. Dựa trên giá trị của thuộc tính, dữ liệu đi theo nhánh tương ứng cho đến khi chạm tới nút lá. Giá trị tại nút lá chính là kết quả dự đoán.

3. Các tiêu chí phân tách (splitting criteria) như Gini Index, Entropy, hay Information Gain được sử dụng trong cây quyết định là gì? Chúng khác nhau ra sao?

Gini Index: $Gini = 1 - \sum_{i=1}^c (p_i)^2$

- Đo mức độ hỗn tạp của dữ liệu, đo lường xác suất một phần tử được chọn ngẫu nhiên bị phân loại sai.
- $Gini = 0 \rightarrow$ hoàn toàn thuần khiết.
- Thường dùng trong thuật toán CART.

Entropy: $H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$

- Đo độ hỗn loạn hoặc không chắc chắn của thông tin.
- Dùng trong ID3, C4.5.

Information Gain (IG): $IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$

- Mức giảm Entropy sau khi chia tách dữ liệu theo một thuộc tính.
- $IG = \text{Entropy}(\text{parent}) - \text{tổng Entropy}(\text{children})$.
- IG cao $\rightarrow$ đặc trưng tách tốt.

Khác nhau:

- Gini tính nhanh hơn, không dùng log.
- Entropy cho phân tách nhạy hơn khi phân bố phức tạp.

- IG dựa trên Entropy, nên mang đặc tính tương tự Entropy.

4. Rừng cây (*Random Forest*) là gì? Nó khác gì so với một cây quyết định đơn lẻ? Tại sao *Random Forest* thường có hiệu suất tốt hơn cây quyết định trong các bài toán phân loại?

Random Forest là một thuật toán học máy tổ hợp (Ensemble Learning) sử dụng phương pháp Bagging (Bootstrap Aggregating). Nó xây dựng nhiều cây quyết định trong quá trình huấn luyện và đưa ra kết quả cuối cùng dựa trên số đông (classification) hoặc trung bình (regression).

Khác biệt:

- Cây đơn: dễ overfit.
- Random Forest: ổn định hơn, giảm nhiễu, hiệu suất cao hơn.

Random Forest thường có hiệu suất tốt hơn cây quyết định:

- Giảm **phương sai (variance)** nhờ trung bình hóa nhiều mô hình.
- Ít nhạy với nhiễu.
- Tự động chọn đặc trưng quan trọng.

5. Những ưu điểm và hạn chế của cây quyết định và *Random Forest* là gì? Trong trường hợp nào thì cây quyết định có thể hoạt động kém hiệu quả?

Cây quyết định

- Ưu điểm: dễ hiểu, trực quan, không yêu cầu chuẩn hóa dữ liệu.
- Hạn chế: dễ overfit; kém ổn định (thay đổi nhỏ trong dữ liệu có thể dẫn đến cây hoàn toàn khác); nhạy với nhiễu.

Random Forest

- Ưu điểm: độ chính xác cao; giảm overfit; hoạt động tốt với dữ liệu lớn.
- Hạn chế: khó giải thích so với cây quyết định; tốn tài nguyên tính toán.

Khi nào cây quyết định kém hiệu quả?

Khi dữ liệu có mối quan hệ phức tạp mà không thể biểu diễn bằng các đường phân chia vuông góc (linear boundaries), hoặc khi dữ liệu quá nhiễu, cây dễ bị mọc quá sâu và học thuộc lòng dữ liệu (overfitting).

6. Viết đoạn code mẫu bằng Python (sử dụng *Scikit-learn*) để xây dựng một mô hình cây quyết định không? Hãy mô tả các bước thực hiện

```

from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
model = tree.DecisionTreeClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)

```

Các bước: chuẩn bị dữ liệu → tách tập → xây mô hình → huấn luyện → đánh giá.

7. Làm thế nào để triển khai một mô hình Random Forest trong Python? Bạn thường thiết lập các tham số nào (ví dụ: `n_estimators`, `max_depth`)?

```

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    max_features="sqrt",
    random_state=0
)
model.fit(X_train, y_train)

```

Để triển khai, sử dụng class `RandomForestClassifier` hoặc `RandomForestRegressor`.

Tham số quan trọng:

- `n_estimators`: số cây.
- `max_depth`: độ sâu tối đa của mỗi cây.
- `max_features`: số đặc trưng dùng khi phân chia.
- `min_samples_split`, `min_samples_leaf`: kiểm soát độ phức tạp.

8. Làm thế nào để đánh giá tầm quan trọng của các đặc trưng (feature importance) trong Random Forest bằng Python?


```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

# Giả sử đã có model 'rf_model' đã được fit và X_train là
DataFrame
# rf_model = RandomForestClassifier(n_estimators=100)
# rf_model.fit(X_train, y_train)

# Lấy giá trị importance
importances = rf_model.feature_importances_

# Tạo DataFrame để hiển thị
feature_imp_df = pd.DataFrame({
    'Feature': ['Feature_A', 'Feature_B', 'Feature_C',
'Feature_D'], # Tên các cột
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

print(feature_imp_df)

# Trực quan hóa
plt.barh(feature_imp_df['Feature'],
feature_imp_df['Importance'])
plt.xlabel("Mức độ quan trọng")
plt.show()

```

Giá trị cao → đặc trưng có ảnh hưởng mạnh.

9. Điều chỉnh siêu tham số (hyperparameter tuning) cho cây quyết định hoặc Random Forest chưa? Hãy mô tả cách bạn sử dụng GridSearchCV hoặc RandomizedSearchCV

```

from sklearn.model_selection import GridSearchCV

# Định nghĩa không gian tham số
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}

rf = RandomForestClassifier(random_state=42)

# Thiết lập Grid Search (cv=5 là cross-validation 5 lần)
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                           cv=5, scoring='accuracy')

# Thực hiện tìm kiếm
grid_search.fit(X_train, y_train)

# Kết quả tốt nhất
print("Tham số tốt nhất:", grid_search.best_params_)
print("Độ chính xác tốt nhất:", grid_search.best_score_)

```

RandomizedSearchCV tương tự nhưng chọn ngẫu nhiên tổ hợp tham số, phù hợp tập tham số lớn.

1.2. BÀI TẬP

Bài tập thực hành 1: Xây dựng cây quyết định và rừng cây trên dữ liệu Titanic

Nhiệm vụ 1: Tải và Tiền xử lý dữ liệu

```

# --- TẢI DỮ LIỆU ---

# Đọc dữ liệu
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')

# Ẩn PassengerId để sau sử dụng trong việc nộp bài
PassengerId = test['PassengerId']

```

```

# --- TIỀN XỬ LÝ DỮ LIỆU ---
# Lưu bản gốc của dữ liệu huấn luyện
original_train = train.copy()

# Kết hợp dữ liệu huấn luyện và kiểm tra để tiền xử lý cùng một lúc
full_data = [train, test]

# Tạo features mới FamilySize là sự kết hợp của SibSp và Parch
for dataset in full_data:
    dataset['FamilySize'] = dataset['SibSp'] + dataset['Parch'] + 1

# Tạo features mới IsAlone từ FamilySize
for dataset in full_data:
    dataset['IsAlone'] = 0
    dataset.loc[dataset['FamilySize'] == 1, 'IsAlone'] = 1

# Xóa tất cả các NULL trong cột Đã lên tàu (Embarked)
for dataset in full_data:
    dataset['Embarked'] = dataset['Embarked'].fillna('S')

# Xóa tất cả các NULL trong cột Fare
for dataset in full_data:
    dataset['Fare'] =
dataset['Fare'].fillna(train['Fare'].median())

# Xử lý các giá trị NULL trong cột Age
for dataset in full_data:
    age_avg = dataset['Age'].mean()
    age_std = dataset['Age'].std()
    age_null_count = dataset['Age'].isnull().sum()
    age_null_random_list = np.random.randint(age_avg - age_std,
    age_avg + age_std, size=age_null_count)

```

```

dataset.loc[np.isnan(dataset['Age']), 'Age'] =
age_null_random_list

    dataset['Age'] = dataset['Age'].astype(int)

# Tạo features Title từ cột Name
def get_title(name):
    title_search = re.search(' ([A-Za-z]+)\.', name)

    if title_search:
        return title_search.group(1)
    return ""

for dataset in full_data:
    dataset['Title'] = dataset['Name'].apply(get_title)

# Nhóm tất cả các "Title" không phổ biến thành một nhóm duy nhất
for dataset in full_data:
    dataset['Title'] = dataset['Title'].replace(['Lady',
'Countess', 'Capt', 'Col', 'Don', 'Dr', 'Major', 'Rev', 'Sir',
'Jonkheer', 'Dona'], 'Rare')

    dataset['Title'] = dataset['Title'].replace('Mlle', 'Miss')
    dataset['Title'] = dataset['Title'].replace('Ms', 'Miss')
    dataset['Title'] = dataset['Title'].replace('Mme', 'Mrs')

# Chuyển đổi các biến phân loại thành các biến số
for dataset in full_data:

    dataset['Sex'] = dataset['Sex'].map( {'female': 0, 'male': 1}
).astype(int)

    title_mapping = {"Mr": 1, "Master": 2, "Mrs": 3, "Miss": 4,
"Rare": 5}

```

```

dataset['Title'] = dataset['Title'].map(title_mapping)
dataset['Title'] = dataset['Title'].fillna(0)

dataset['Embarked'] = dataset['Embarked'].map( {'S': 0, 'C':
1, 'Q': 2} ).astype(int)

dataset.loc[ dataset['Fare'] <= 7.91, 'Fare']
            = 0

dataset.loc[(dataset['Fare'] > 7.91) & (dataset['Fare'] <=
14.454), 'Fare'] = 1

dataset.loc[(dataset['Fare'] > 14.454) & (dataset['Fare'] <=
31), 'Fare']    = 2

dataset.loc[ dataset['Fare'] > 31, 'Fare']
            = 3

dataset['Fare'] = dataset['Fare'].astype(int)

dataset.loc[ dataset['Age'] <= 16, 'Age']
            = 0

dataset.loc[(dataset['Age'] > 16) & (dataset['Age'] <= 32),
'Age'] = 1

dataset.loc[(dataset['Age'] > 32) & (dataset['Age'] <= 48),
'Age'] = 2

dataset.loc[(dataset['Age'] > 48) & (dataset['Age'] <= 64),
'Age'] = 3

dataset.loc[ dataset['Age'] > 64, 'Age'] ;

```

1. Tạo biến FamilySize và IsAlone

- FamilySize được tính bằng tổng SibSp và Parch cộng thêm 1.
- IsAlone = 1 khi FamilySize = 1, ngược lại = 0.
- Mục đích: biểu diễn quy mô gia đình đi cùng hành khách.

2. Xử lý giá trị khuyết (NULL)

- Cột Embarked: thay thế giá trị thiếu bằng 'S'.

- Cột Fare: thay thế giá trị thiếu bằng median của Fare trong tập train.
 - Cột Age: giá trị thiếu được thay thế bằng các số ngẫu nhiên trong khoảng ($\text{mean} \pm \text{std}$) của Age trong tập train.
3. Trích xuất Title từ cột Name
 - Dùng regex để lấy danh xưng (Mr, Mrs, Miss, Master, v.v.) từ chuỗi Name.
 - Gom nhóm các danh xưng ít xuất hiện vào một nhóm chung “Rare”.
 4. Chuyển đổi biến phân loại sang dạng số
 - Sex: female $\rightarrow$ 0, male $\rightarrow$ 1.
 - Title: ánh xạ Mr = 1, Master = 2, Miss = 3, Mrs = 4, Rare = 5.
 - Embarked: S = 0, C = 1, Q = 2.
 5. Rời rạc hóa (binning) Fare và Age
 - Fare được chia thành 4 khoảng dựa trên các mức giá trị cụ thể trong code.
 - Age cũng được chia thành 4 khoảng giá trị (0–16, 17–32, 33–48, 49–64, >64).
 6. Mục tiêu của các bước trên
 - Chuẩn hóa dữ liệu đầu vào, giảm nhiễu từ giá trị thiếu.
 - Tạo thêm đặc trưng có ý nghĩa cho mô hình (FamilySize, IsAlone, Title).
 - Chuyển dữ liệu thành dạng mô hình học máy dễ xử lý (mã hóa và rời rạc hóa).

```
# --- CHIA DỮ LIỆU TRAIN/TEST ---
y_train = train['Survived']
x_train = train.drop(['Survived'], axis=1).values
x_test = test.values

print(f"Kích thước tập train: {x_train.shape}")
print(f"Kích thước tập test: {x_test.shape}")
```

Kích thước tập train: (891, 9)

Kích thước tập test: (418, 9)

Nhiệm Vụ 2: Xây Dựng Mô Hình Cây Quyết Định

```
# --- MÔ HÌNH CÂY QUYẾT ĐỊNH (DECISION TREE) ---
print("\n--- KẾT QUẢ DECISION TREE ---")
# Khởi tạo mô hình (giới hạn độ sâu để tránh overfitting)
dt_model = tree.DecisionTreeClassifier(max_depth=4)
dt_model.fit(x_train, y_train)
```

--- KẾT QUẢ DECISION TREE ---

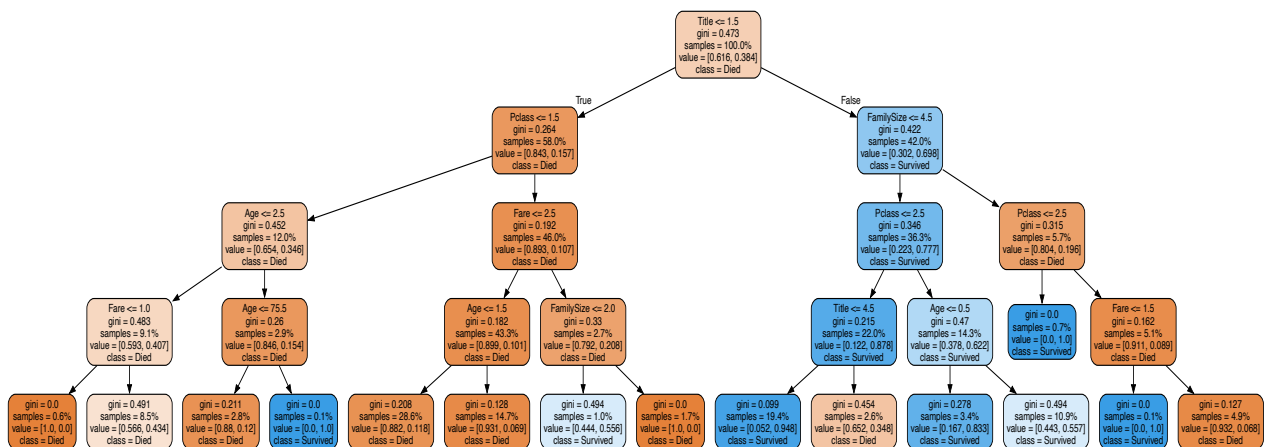
DecisionTreeClassifier		
▼ Parameters		
	criterion	'gini'
	splitter	'best'
	max_depth	4
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	None
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	class_weight	None
	ccp_alpha	0.0
	monotonic_cst	None

Nhận Xét:

- Mô hình sử dụng DecisionTreeClassifier với tham số chính là $\text{max_depth} = 4$. Đây là tham số duy nhất được thay đổi so với giá trị mặc định trong bảng tham số.
- **Ý nghĩa lựa chọn $\text{max_depth} = 4$:**
 - Giới hạn độ sâu giúp mô hình giảm nguy cơ overfitting.
 - Cây ở độ sâu 4 vẫn đủ khả năng tạo ra các nút phân chia chính mà không quá phức tạp.
- **Kết luận về mô hình sau khi huấn luyện:**
 - Đây là mô hình cây quyết định có độ sâu trung bình, không quá cạn (dễ underfitting) và không quá sâu (dễ overfitting).
 - Toàn bộ siêu tham số còn lại đều đang giữ mặc định nên mô hình tập trung chủ yếu vào cấu trúc cây do max_depth quyết định.

```
# --- HIỂN THỊ CÂY QUYẾT ĐỊNH (DECISION TREE) ---
dot_data = tree.export_graphviz(dt_model,
                                out_file=None,
                                max_depth=4,
                                filled=True,
                                rounded=True,
                                impurity = True,
                                feature_names=list(train.drop(['Survived'], axis=1)),
                                proportion=True,
                                class_names=['Died', 'Survived'])

graph = graphviz.Source(dot_data)
graph
```



Nhận Xét:

Nút gốc sử dụng Title (≤ 1.5) là đặc trưng quan trọng nhất. Nhánh trái tập trung mẫu Died, nhánh phải tập trung mẫu Survived. Các thuộc tính phân chia tiếp theo: Pclass, Age, Fare, FamilySize ở tầng 2-3. Nút lá có Gini thấp (nhiều nút = 0.0), phân lớp thuần khiết. Cây đạt max_depth = 4 và cân bằng giữa hai nhánh.

```
print(f"\nĐộ chính xác trên tập huấn luyện: ")
acc_decision_tree = round(dt_model.score(x_train, y_train) * 100, 2)
print(acc_decision_tree, "%")
```

Độ chính xác trên tập huấn luyện:
83.5 %

Đánh giá hiệu suất Decision Tree trên tập huấn luyện:

Độ chính xác 83.5% cho thấy mô hình phân loại đúng 83.5% mẫu trong tập huấn luyện. Kết quả này chấp nhận được nhưng không quá cao, cho thấy mô hình không bị overfitting nghiêm trọng do giới hạn `max_depth = 4`. Cần đánh giá trên tập test để xác định khả năng tổng quát hóa thực tế.

```
# --- TẠO CÂY QUYẾT ĐỊNH (DECISION TREE) VÀ TÌM THAM SỐ TỐI ƯU BẰNG GRIDSEARCHCV ---
params = {'max_depth': [1, 2, 4, 6, 8, 10, 12]} # các thông số
dt = tree.DecisionTreeClassifier() # khởi tạo mô hình cây quyết định
cv = GridSearchCV(dt, param_grid=params, scoring='roc_auc', # đánh giá bằng ROC AUC
n_jobs=None, refit=True, cv=4, verbose=1,
error_score=np.nan,
return_train_score=True) # Trả về tham số tối ưu để mô hình tốt nhất
cv.fit(x_train, y_train)
```

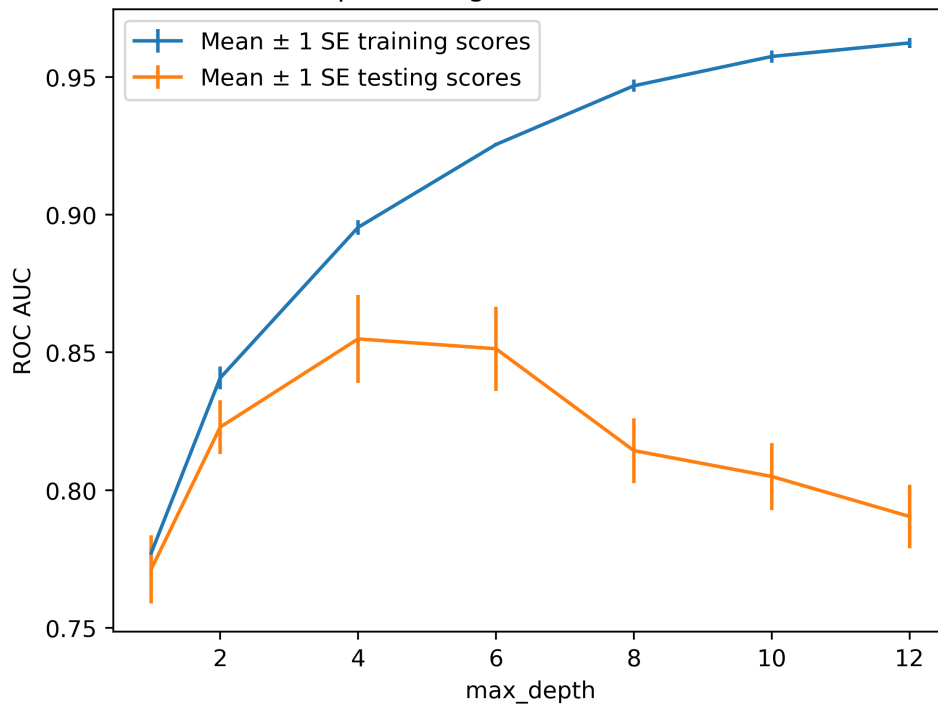
Fitting 4 folds for each of 7 candidates, totalling 28 fits

GridSearchCV		
▼ Parameters		
estimator	DecisionTreeClassifier()	
param_grid	{ 'max_depth': [1, 2, ...] }	
scoring	'roc_auc'	
n_jobs	None	
refit	True	
cv	4	
verbose	1	
pre_dispatch	'2*n_jobs'	
error_score	nan	
return_train_score	True	

best_estimator_: DecisionTreeClassifier		
DecisionTreeClassifier(max_depth=4)		
▼ Parameters		
criterion	'gini'	
splitter	'best'	
max_depth	4	
min_samples_split	2	
min_samples_leaf	1	
min_weight_fraction_leaf	0.0	
max_features	None	
random_state	None	
max_leaf_nodes	None	
min_impurity_decrease	0.0	
class_weight	None	
ccp_alpha	0.0	
monotonic_cst	None	

```
# --- VẼ BIỂU ĐỒ SO SÁNH KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH ---
cv_results_df = pd.DataFrame(cv.cv_results_)
cv_results_df.columns
ax = plt.axes()
ax.errorbar(cv_results_df['param_max_depth'],
cv_results_df['mean_train_score'],
yerr=cv_results_df['std_train_score']/np.sqrt(4),
label='Mean  $\pm$  1 SE training scores')
ax.errorbar(cv_results_df['param_max_depth'],
cv_results_df['mean_test_score'],
yerr=cv_results_df['std_test_score']/np.sqrt(4),
label='Mean  $\pm$  1 SE testing scores')
ax.legend()
plt.title('Biểu đồ so sánh kết quả đánh giá mô hình với các tham số khác nhau')
plt.xlabel('max_depth')
plt.ylabel('ROC AUC')
```

Biểu đồ so sánh kết quả đánh giá mô hình với các tham số khác nhau



Phân tích biểu đồ ROC AUC theo max_depth:

- Training scores (xanh): Tăng liên tục từ ~0.77 đến ~0.96 khi max_depth tăng từ 1 đến 12. Đường cong đi lên cho thấy mô hình học tốt hơn trên tập huấn luyện với độ sâu lớn hơn.
- Testing scores (cam): Đạt đỉnh ~0.87 tại max_depth = 4-6, sau đó giảm dần xuống ~0.79 tại max_depth = 12.

- Overfitting: Khoảng cách giữa hai đường tăng mạnh từ $\text{max_depth} \geq 6$, chênh lệch lên đến ~ 0.17 tại $\text{max_depth} = 12$. Mô hình bị overfitting nghiêm trọng khi cây quá sâu.

Kết luận: $\text{max_depth} = 4$ là lựa chọn tối ưu, cân bằng giữa hiệu suất training (0.90) và testing (0.85-0.87), tránh overfitting.

Nhiệm Vụ 3: Mô hình Rừng Cây Ngẫu Nhiên

--- KHỞI TẠO MÔ HÌNH RỪNG CÂY ---

```
rf = RandomForestClassifier(n_estimators=100, criterion='gini', max_depth=5, min_samples_split=2,
                           min_samples_leaf=1, min_weight_fraction_leaf=0.0,
                           max_features='sqrt', max_leaf_nodes=None, min_impurity_decrease=0.0,
                           bootstrap=True, oob_score=False, n_jobs=None,
                           random_state=42, verbose=0, warm_start=False, class_weight=None)
```

--- TÌM VÀ TRAIN THAM SỐ TỐI ƯU CHO MÔ HÌNH RỪNG CÂY ---

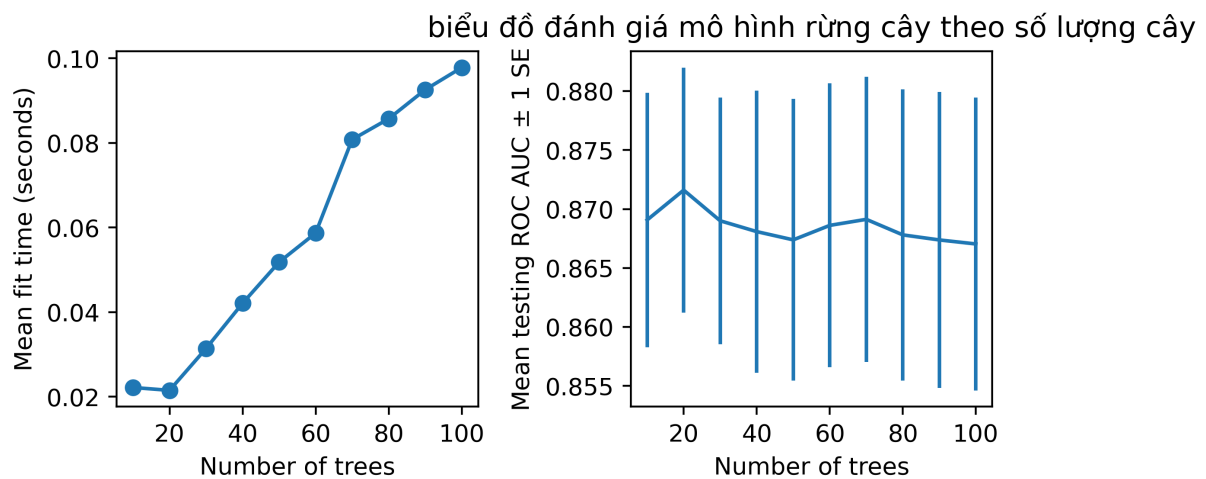
```
#lưới tham số cho bài tập này để tìm kiếm số lượng cây, trong phạm vi từ 10 đến 100 theo đơn vị 10 giây
rf_params_ex = {'n_estimators':list(range(10,110,10))}
cv_rf_ex = GridSearchCV(rf, param_grid=rf_params_ex,
                        scoring='roc_auc', n_jobs=None, # sử dụng tất cả CPU có sẵn
                        refit=True, cv=4, verbose=1, # hiển thị tiến trình
                        error_score=np.nan, # xử lý lỗi
                        return_train_score=True) # Trả về tham số tối ưu để mô hình tốt nhất
cv_rf_ex.fit(x_train, y_train)
```

Fitting 4 folds for each of 10 candidates, totalling 40 fits

GridSearchCV	
▼ Parameters	
estimator	RandomForestC...ndom_state=42)
param_grid	{'n_estimators': [10, 20, ...]}
scoring	'roc_auc'
n_jobs	None
refit	True
cv	4
verbose	1
pre_dispatch	'2*n_jobs'
error_score	nan
return_train_score	True

best_estimator_: RandomForestClassifier	
RandomForestClassifier(max_depth=5, n_estimators=20, random_state=42)	
▼ Parameters	
n_estimators	20
criterion	'gini'
max_depth	5
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

```
# --- BIỂU ĐỒ ĐÁNH GIÁ MÔ HÌNH RỪNG CÂY ---
cv_rf_ex_results_df = pd.DataFrame(cv_rf_ex.cv_results_) # chuyển kết quả sang DataFrame
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
axs[0].plot(cv_rf_ex_results_df['param_n_estimators'], # số lượng cây
            cv_rf_ex_results_df['mean_fit_time'], # thời gian huấn luyện trung bình
            '-o')
axs[0].set_xlabel('Number of trees')
axs[0].set_ylabel('Mean fit time (seconds)')
axs[1].errorbar(cv_rf_ex_results_df['param_n_estimators'],
               cv_rf_ex_results_df['mean_test_score'], # đánh giá ROC AUC trung bình trên tập kiểm tra
               yerr=cv_rf_ex_results_df['std_test_score']/np.sqrt(4)) # sai số chuẩn
axs[1].set_xlabel('Number of trees')
axs[1].set_ylabel('Mean testing ROC AUC  $\pm$  1 SE ')
plt.title('biểu đồ đánh giá mô hình rừng cây theo số lượng cây')
plt.tight_layout()
```

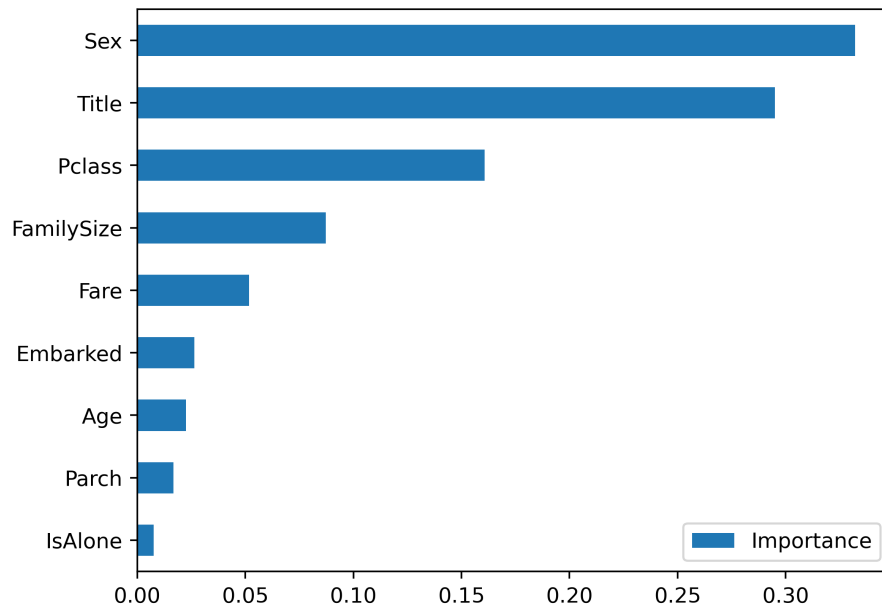


Nhận xét:

- **Biểu đồ trái - Thời gian huấn luyện:** Tăng tuyến tính từ ~0.02s (20 cây) đến ~0.10s (100 cây). Mỗi cây thêm vào làm tăng thời gian huấn luyện tỷ lệ thuận.
- **Biểu đồ phải - ROC AUC testing:** Dao động trong khoảng 0.855-0.880 với sai số chuẩn (± 1 SE) lớn ở mọi số lượng cây. Hiệu suất tương đối ổn định, không cải thiện rõ rệt khi tăng số cây từ 20 lên 100. Giá trị trung bình giảm nhẹ từ ~0.872 xuống ~0.868.

Kết luận: Tăng số cây từ 20 lên 100 không mang lại lợi ích đáng kể về độ chính xác (ROC AUC gần như không đổi) nhưng tăng đáng kể thời gian huấn luyện (x5 lần). Số lượng cây từ 20-40 là lựa chọn hợp lý để cân bằng giữa hiệu suất và chi phí tính toán.

```
# --- HIỂN THỊ KẾT QUẢ VÀ TẦM QUAN TRỌNG CỦA CÁC ĐẶC TRƯNG ---
cv_rf_ex.best_params_
feat_imp_df = pd.DataFrame({
    'Importance': cv_rf_ex.best_estimator_.feature_importances_,
    index=train.drop(['Survived'], axis=1).columns)
feat_imp_df.sort_values('Importance', ascending=True).plot.barh()
```



Kết luận: Sex và Title chiếm ~62% tổng importance, là yếu tố quyết định chính. Các đặc trưng xã hội-kinh tế (Pclass, Fare) và gia đình (FamilySize) có ảnh hưởng thứ yếu. Age có vai trò nhỏ bất ngờ so với dự đoán ban đầu.

Bài tập thực hành 2: Xây dựng cây quyết định và rừng cây trên dữ liệu bệnh tiểu đường.

Nhiệm Vụ 1: Tải và Tiền Xử Lý Dữ Liệu

```
# --- TẢI DỮ LIỆU ---
df = pd.read_csv('diabetes_prediction_dataset.csv')
```

```
# KIỂM TRA VÀ XỬ LÝ DỮ LIỆU THIẾU, TRÙNG

# Xóa giá trị không cần thiết [0.00195%]
df = df[df['gender'] != 'Other']

# Kiểm tra giá trị Null
has_null = df.isnull().values.any()
print("Có giá trị Null:", has_null)

# Kiểm tra giá trị NaN
has_nan = df.isna().values.any()
print("Có giá trị NaN:", has_nan)
```

Có giá trị Null: False

Có giá trị NaN: False

```
# --- TIỀN XỬ LÝ DỮ LIỆU CHUYÊN SÂU ---
# Kiểm tra các giá trị duy nhất trong cột 'smoking_history'
print("\nGiá trị duy nhất trong cột 'smoking_history':")
print(df['smoking_history'].value_counts())

# Hàm để tái phân loại trạng thái hút thuốc
def recategorize_smoking(smoking_status):
    if smoking_status in ['never', 'No Info']:
        return 'non-smoker'
    elif smoking_status == 'current':
        return 'current'
    elif smoking_status in ['ever', 'former', 'not current']:
        return 'past_smoker'

# Áp dụng hàm tái phân loại
df['smoking_history'] = df['smoking_history'].apply(recategorize_smoking)

# Kiểm tra lại các giá trị duy nhất sau khi tái phân loại
print("\nGiá trị duy nhất trong cột 'smoking_history' sau khi tái phân loại:")
print(df['smoking_history'].value_counts())
```

Giá trị duy nhất trong cột 'smoking_history':

```
smoking_history
No Info      35810
never        35092
former       9352
current      9286
not current   6439
ever         4003
Name: count, dtype: int64
```

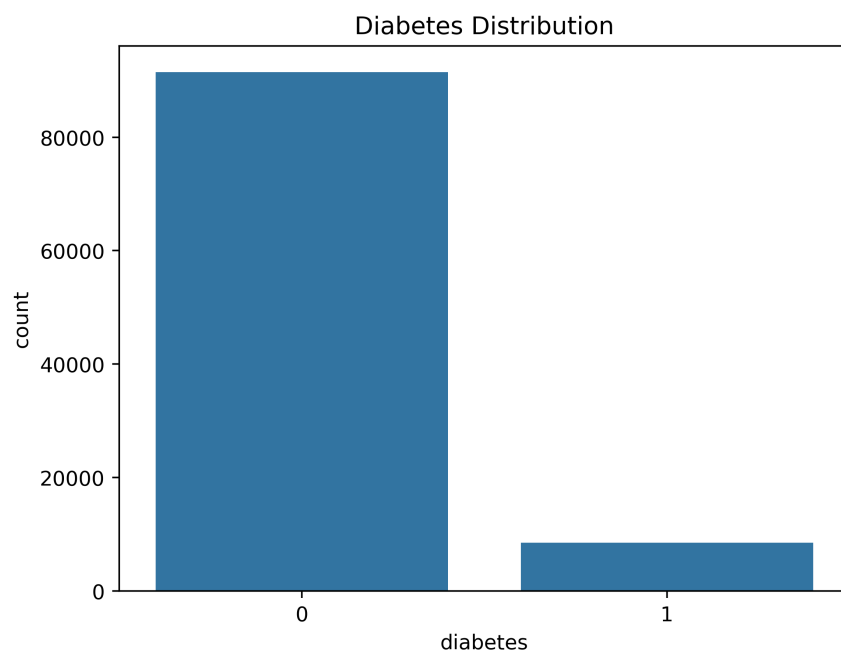
Giá trị duy nhất trong cột 'smoking_history' sau khi tái phân loại:

```
smoking_history
non-smoker    70902
past_smoker   19794
current       9286
Name: count, dtype: int64
```

```
# Mã hóa biến danh mục
df['gender'] = df['gender'].map({'Female': 0, 'Male': 1})

smoking_map = {'non-smoker': 1, 'past_smoker': 2, 'current': 3}
df['smoking_history'] = df['smoking_history'].map(smoking_map).fillna(0).astype(int)
```

```
# Biểu đồ đếm cho biến 'diabetes'
sns.countplot(x='diabetes', data=df)
plt.title('Diabetes Distribution')
plt.show()
```



Phân bố mất cân bằng lớp trong biến Diabetes:

- Lớp 0 (không bị tiểu đường): ~92,000 mẫu
- Lớp 1 (bị tiểu đường): ~8,000 mẫu

```
# --- CHIA DỮ LIỆU TRAIN/TEST ---
# Chia dữ liệu thành biến đầu vào X và biến mục tiêu y
X = data.drop('diabetes', axis=1)
y = data['diabetes']

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, stratify=y, random_state=42)

print(f"\nKích thước tập Train: {X_train.shape}")
print(f"Kích thước tập Test: {X_test.shape}")
```

Kích thước tập Train: (79985, 8)
 Kích thước tập Test: (19997, 8)

Nhiệm Vụ 2: Mô hình Cây Quyết Định

```
# --- MÔ HÌNH CÂY QUYẾT ĐỊNH (DECISION TREE) ---
print("\n--- KẾT QUẢ DECISION TREE ---")
# Khởi tạo mô hình (giới hạn độ sâu để tránh overfitting)
dt_model = tree.DecisionTreeClassifier(max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)
```

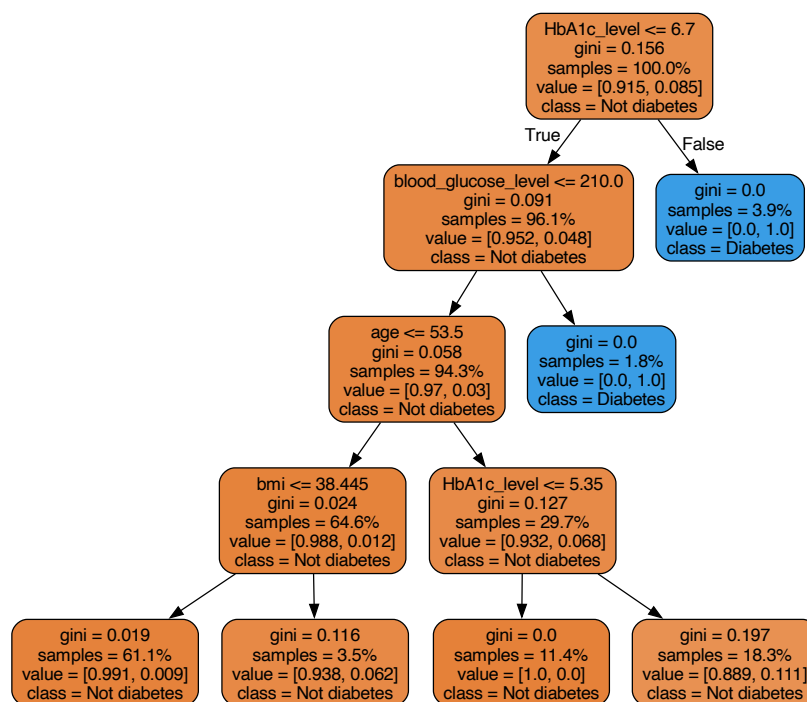
✓ 0.0s

--- KẾT QUẢ DECISION TREE ---

DecisionTreeClassifier		
▼ Parameters		
	criterion	'gini'
	splitter	'best'
	max_depth	4
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	42
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	class_weight	None
	ccp_alpha	0.0
	monotonic_cst	None

Nhận xét: Cấu hình đơn giản, tập trung vào kiểm soát overfitting thông qua max_depth. Không xử lý mất cân bằng lớp - nên cân nhắc thêm class_weight='balanced' cho dữ liệu Diabetes.


```
# --- HIỂN THỊ CÂY QUYẾT ĐỊNH (DECISION TREE) ---
dot_data = tree.export_graphviz(dt_model,
                                out_file=None,
                                filled=True,
                                rounded=True,
                                feature_names=\
X_train_resampled.columns,
                                proportion=True,
                                class_names=['Not diabetes', 'Diabetes'])
graph = graphviz.Source(dot_data)
graph.render("decision_tree_diabetes")
graph
```



Phân tích cấu trúc Decision Tree dự đoán Diabetes:

- **Nút gốc:** $\text{HbA1c_level} \leq 6.7$ là đặc trưng quan trọng nhất. Gini = 0.156 cho thấy phân bố lệch mạnh về lớp "Not diabetes" (91.5% vs 8.5%).
- **Nhánh trái ($\text{HbA1c} \leq 6.7$):** Chiếm 96.1% dữ liệu, phân chia tiếp theo $\text{blood_glucose_level} \leq 210.0$, sau đó $\text{age} \leq 53.5$ và $\text{bmi} \leq 38.445$. Đa số nút lá (màu cam) thuộc lớp "Not diabetes" với Gini thấp (0.019-0.116), cho thấy độ thuần khiết cao.
- **Nhánh phải ($\text{HbA1c} > 6.7$):** Chỉ chiếm 3.9% dữ liệu nhưng 100% là lớp "Diabetes" (Gini = 0.0). Đây là nhánh quyết định rõ ràng nhất - HbA1c cao là dấu hiệu chắc chắn của tiểu đường.

- **Đặc trưng phân chia:** HbA1c_level, blood_glucose_level, age, bmi - phù hợp với y học lâm sàng.

Vấn đề: Cây thiên về dự đoán "Not diabetes" do mất cân bằng lớp (91.5:8.5). Nhánh phải tuy chính xác 100% nhưng chỉ bắt được 3.9% dữ liệu.

```
acc_decision_tree = round(dt_model.score(X_train, y_train) * 100, 2)
acc_decision_tree
```

✓ 0.0s

97.18

Mô hình phân loại đúng 97.18% mẫu trong tập huấn luyện. Độ chính xác cao này có thể do:

1. **Mất cân bằng lớp nghiêm trọng (91.5:8.5):** Mô hình dễ đạt accuracy cao bằng cách dự đoán đúng lớp đa số "Not diabetes"
2. **Overfitting tiềm ẩn:** Accuracy 97.18% trên training có thể không phản ánh hiệu suất thực tế trên test set
3. **Metric không phù hợp:** Với dữ liệu mất cân bằng, accuracy không đánh giá tốt khả năng phát hiện lớp thiểu số "Diabetes"

```
# --- TẠO CÂY QUYẾT ĐỊNH (DECISION TREE) VÀ TÌM THAM SỐ TỐI ƯU BẰNG GRIDSEARCHCV ---
params = {'max_depth':[1, 2, 4, 6, 8, 10, 12]} #các thông số
dt = tree.DecisionTreeClassifier() # khởi tạo mô hình cây quyết định
cv = GridSearchCV(dt, param_grid=params, scoring='roc_auc', # đánh giá bằng ROC AUC
n_jobs=None, refit=True, cv=4, verbose=1,
error_score=np.nan,
return_train_score=True) # Trả về tham số tối ưu để mô hình tốt nhất
cv.fit(X_train, y_train)
```

Fitting 4 folds for each of 7 candidates, totalling 28 fits

GridSearchCV		
▼ Parameters		
estimator		DecisionTreeClassifier()
param_grid		{'max_depth': [1, 2, ...]}
scoring		'roc_auc'
n_jobs		None
refit		True
cv		4
verbose		1
pre_dispatch		'2*n_jobs'
error_score		nan
return_train_score		True
▼ best_estimator_: DecisionTreeClassifier		
DecisionTreeClassifier(max_depth=8)		

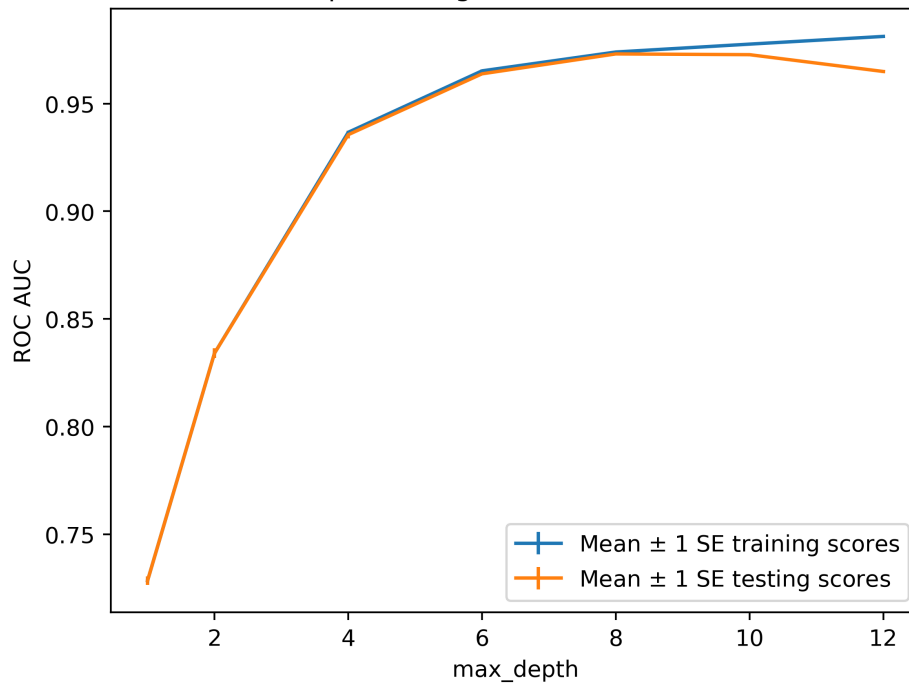
DecisionTreeClassifier		
▼ Parameters		
criterion		'gini'
splitter		'best'
max_depth		8
min_samples_split		2
min_samples_leaf		1
min_weight_fraction_leaf		0.0
max_features		None
random_state		None
max_leaf_nodes		None
min_impurity_decrease		0.0
class_weight		None
ccp_alpha		0.0
monotonic_cst		None

Nhận xét:

GridSearchCV chọn max_depth = 8 dựa trên ROC AUC, phù hợp hơn accuracy cho dữ liệu mất cân bằng. Metric ROC AUC đánh giá tốt khả năng phân biệt giữa hai lớp, không bị ảnh hưởng bởi tỷ lệ lớp. Cần so sánh ROC AUC giữa training và validation để xác nhận không overfitting.

```
# --- VẼ BIỂU ĐỒ SO SÁNH KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH ---
cv_results_df = pd.DataFrame(cv.cv_results_)
cv_results_df.columns
ax = plt.axes()
ax.errorbar(cv_results_df['param_max_depth'],
cv_results_df['mean_train_score'],
yerr=cv_results_df['std_train_score']/np.sqrt(4),
label='Mean  $\pm$  1 SE training scores')
ax.errorbar(cv_results_df['param_max_depth'],
cv_results_df['mean_test_score'],
yerr=cv_results_df['std_test_score']/np.sqrt(4),
label='Mean  $\pm$  1 SE testing scores')
ax.legend()
plt.title('Biểu đồ so sánh kết quả đánh giá mô hình với các tham số khác nhau')
plt.xlabel('max_depth')
plt.ylabel('ROC AUC')
```

Biểu đồ so sánh kết quả đánh giá mô hình với các tham số khác nhau



Nhận xét:

- **Rủi ro thiên kiến:** Dữ liệu mất cân bằng nghiêm trọng (tỷ lệ 9:1) kết hợp với `class_weight=None` sẽ khiến mô hình bỏ sót các trường hợp bị bệnh (nhãn 1).
- **Chưa tối ưu hóa (Underfitting):** Cấu hình hiện tại tại `max_depth=4` chưa khai thác hết dữ liệu, hiệu suất thấp hơn mức tiềm năng.
- **Điểm tối ưu thực tế:** Biểu đồ xác nhận `max_depth=8` là điểm tốt nhất (ROC AUC ~0.975); vượt quá ngưỡng này mô hình sẽ bị quá khớp (Overfitting).

Nhiệm Vụ 2: Mô hình Rừng cây ngẫu nhiên

```
# --- KHỞI TẠO MÔ HÌNH RỪNG CÂY ---
rf = RandomForestClassifier\
(n_estimators=100, criterion='gini', max_depth=5,
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_features='sqrt', max_leaf_nodes=None, min_impurity_decrease=0.0,
bootstrap=True, oob_score=False, n_jobs=None,
random_state=42, verbose=0, warm_start=False, class_weight=None)
```

```
# --- TÌM VÀ TRAIN THAM SỐ TỐI ƯU CHO MÔ HÌNH RỪNG CÂY ---
#lưới tham số cho bài tập này để tìm kiếm số lượng cây, trong phạm vi từ 10 đến 100 theo đơn vị 10 giây
rf_params_ex = {'n_estimators':list(range(10,110,10))}
cv_rf_ex = GridSearchCV(rf, param_grid=rf_params_ex,
                        scoring='roc_auc', n_jobs=None, # sử dụng tất cả CPU có sẵn
                        refit=True, cv=4, verbose=1, # hiển thị tiến trình
                        error_score=np.nan, # xử lý lỗi
                        return_train_score=True) # Trả về tham số tối ưu để mô hình tốt nhất
cv_rf_ex.fit(X_train_resampled, y_train_resampled)
```

Fitting 4 folds for each of 10 candidates, totalling 40 fits

▼

GridSearchCV

Parameters

	estimator	RandomForestC...ndom_state=42)
	param_grid	{ 'n_estimators': [10, 20, ...]}
	scoring	'roc_auc'
	n_jobs	None
	refit	True
	cv	4
	verbose	1
	pre_dispatch	'2*n_jobs'
	error_score	nan
	return_train_score	True

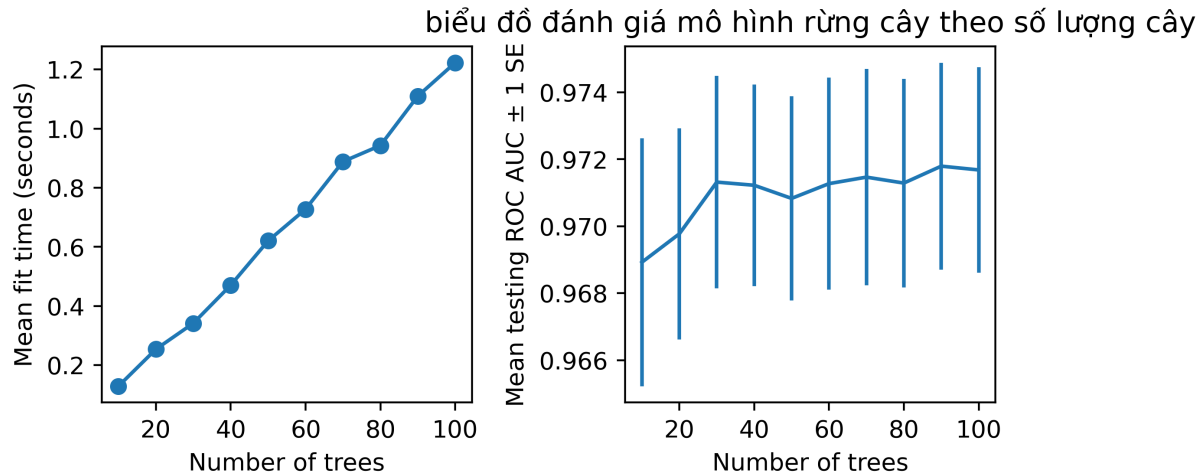
▼

best_estimator_: RandomForestClassifier

RandomForestClassifier(max_depth=5, n_estimators=90, random_state=42)

RandomForestClassifier		
▼ Parameters		
n_estimators	90	
criterion	'gini'	
max_depth	5	
min_samples_split	2	
min_samples_leaf	1	
min_weight_fraction_leaf	0.0	
max_features	'sqrt'	
max_leaf_nodes	None	
min_impurity_decrease	0.0	
bootstrap	True	
oob_score	False	
n_jobs	None	
random_state	42	
verbose	0	
warm_start	False	
class_weight	None	
ccp_alpha	0.0	
max_samples	None	
monotonic_cst	None	

```
# --- BIỂU ĐỒ ĐÁNH GIÁ MÔ HÌNH RỪNG CÂY ---
cv_rf_ex_results_df = pd.DataFrame(cv_rf_ex.cv_results_) # chuyển kết quả sang DataFrame
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
axs[0].plot(cv_rf_ex_results_df['param_n_estimators'], # số lượng cây
            cv_rf_ex_results_df['mean_fit_time'], # thời gian huấn luyện trung bình
            '-o')
axs[0].set_xlabel('Number of trees')
axs[0].set_ylabel('Mean fit time (seconds)')
axs[1].errorbar(cv_rf_ex_results_df['param_n_estimators'],
               cv_rf_ex_results_df['mean_test_score'], # đánh giá ROC AUC trung bình trên tập kiểm tra
               yerr=cv_rf_ex_results_df['std_test_score']/np.sqrt(4)) # sai số chuẩn
axs[1].set_xlabel('Number of trees')
axs[1].set_ylabel('Mean testing ROC AUC $\pm$ 1 SE ')
plt.title('biểu đồ đánh giá mô hình rừng cây theo số lượng cây')
plt.tight_layout()
```



Hiệu suất bão hòa sớm (Diminishing Returns):

- Chỉ số ROC AUC trên tập kiểm tra tăng nhanh từ mức ~0.969 (10 cây) lên mức ổn định ~0.9715 tại ngưỡng **30 cây**.
- Từ 40 đến 100 cây, đường trung bình ROC AUC đi ngang (plateau) và dao động nhẹ quanh mức 0.971 - 0.972. Việc thêm cây sau mốc 40 không tạo ra sự cải thiện rõ rệt về độ chính xác dự báo.

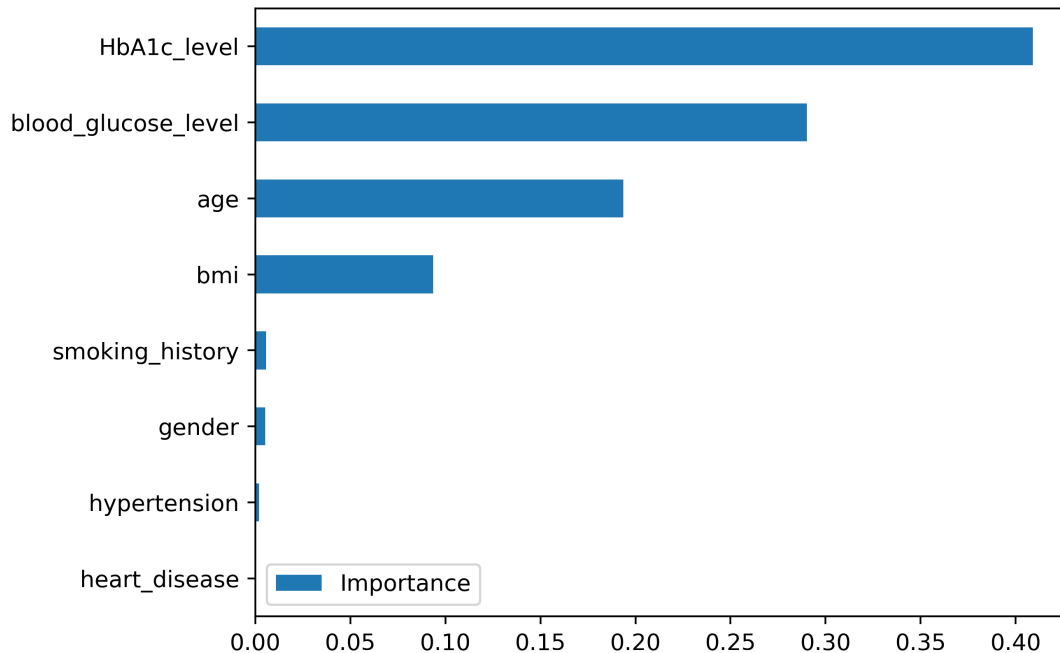
Chi phí tính toán tăng tuyến tính:

- Biểu đồ bên trái cho thấy thời gian huấn luyện (Mean fit time) tỉ lệ thuận hoàn toàn với số lượng cây.
- Huấn luyện 100 cây tốn khoảng 1.2 giây, trong khi 30 cây chỉ tốn khoảng 0.35 giây.

Kết luận về cấu hình tối ưu:

- Điểm tối ưu nằm ở khoảng **30 đến 40 cây**. Tại đây, mô hình đạt hiệu suất cao nhất (~0.971) với thời gian huấn luyện thấp.
- Cấu hình 100 cây không hiệu quả về mặt kinh tế: tốn gấp 3-4 lần thời gian tính toán so với mốc 30 cây nhưng kết quả đầu ra gần như tương đương.

```
# --- HIỂN THỊ KẾT QUẢ VÀ TẦM QUAN TRỌNG CỦA CÁC ĐẶC TRƯNG ---
cv_rf_ex.best_params_
feat_imp_df = pd.DataFrame({
    'Importance':cv_rf_ex.best_estimator_.feature_importances_,
    index=X_train_resampled.columns)
feat_imp_df.sort_values('Importance', ascending=True).plot.barh()
```



Đánh giá:

- xét nghiệm HbA1c và đường huyết lúc đói chính là "**tiêu chuẩn vàng**" để chẩn đoán một người có bị tiểu đường hay không.
- Tuổi tác và Béo phì (BMI cao) là các yếu tố **nguy cơ hàng đầu** (risk factors) dẫn đến tiểu đường loại 2. Tuy nhiên, chúng là *nguyên nhân* hoặc *điều kiện thúc đẩy*, chứ không phải là *biểu hiện trực tiếp* của bệnh như lượng đường trong máu. Do đó, mức độ quan trọng của chúng thấp hơn nhóm xét nghiệm máu.
- Dùng Age và BMI (cân nặng/chiều cao) để khoanh vùng nhóm nguy cơ cao. Đây là các thông tin dễ thu thập, không tốn kém.
- Với nhóm nguy cơ cao, chỉ cần tập trung xét nghiệm HbA1c hoặc Blood Glucose. Các thông tin khác như lịch sử hút thuốc hay giới tính hầu như không giúp ích nhiều cho việc khẳng định bệnh trong mô hình này.

PHẦN II: SUPPORT VECTOR MACHINE (SVM)

2.1. ÔN TẬP LÝ THUYẾT

1. SVM hoạt động như thế nào? Giải thích hyperplane và margin

Hyperplane: Là siêu phẳng dùng để phân tách các lớp dữ liệu. Với dữ liệu 2 chiều, hyperplane là một đường thẳng; với 3 chiều là mặt phẳng; nhiều chiều hơn là siêu phẳng.

Margin: Khoảng cách từ hyperplane đến các điểm gần nhất của mỗi lớp. Mục tiêu của SVM là tìm hyperplane có **margin lớn nhất**, giúp mô hình tổng quát hóa tốt hơn.

Cách hoạt động của SVM:

- SVM tìm ra hyperplane tối ưu sao cho **tối đa hóa margin** giữa hai lớp.
- Nếu dữ liệu không tách tuyến tính được, SVM dùng **kernel trick** để ánh xạ dữ liệu sang không gian có số chiều cao hơn, nơi dữ liệu có thể tách tuyến tính.
- Chỉ các điểm gần hyperplane nhất quyết định vị trí của nó.

2. Support Vectors là gì? Vai trò của chúng

Support vectors là những điểm dữ liệu nằm gần hyperplane nhất.

Vai trò:

- Chúng **quyết định vị trí và hướng của hyperplane**.
- Nếu bỏ các điểm không phải support vectors, hyperplane không thay đổi.
- Nếu thay đổi hoặc nhiều trong support vectors, hyperplane sẽ thay đổi đáng kể.

Do đó chúng **cực kỳ quan trọng** và là cốt lõi trong SVM.

3. Khác biệt giữa Hard Margin và Soft Margin. Khi nào dùng soft margin?

Hard Margin:

- Không cho phép tồn tại lỗi phân loại.
- Chỉ dùng khi dữ liệu **tách tuyến tính hoàn toàn**.
- Nhạy với nhiễu và outliers.

Soft Margin:

- Cho phép vi phạm một chút (cho phép một số điểm nằm sai phía hyperplane).
- Điều khiển bởi tham số C .
- Thực tế dùng nhiều hơn.

Khi nào dùng soft margin?

- Khi dữ liệu có nhiễu.
- Khi dữ liệu không hoàn toàn tách tuyến tính.
- Khi muốn ưu tiên tổng quát hóa thay vì phân loại hoàn hảo.

4. Kernel trong SVM là gì? Các loại kernel phổ biến và khi nào dùng?

Kernel là hàm giúp ánh xạ dữ liệu từ không gian ban đầu sang không gian mới có số chiều cao hơn, để dữ liệu có thể tách tuyến tính.

Các kernel phổ biến:

1. **Linear kernel:**
 - Dùng khi dữ liệu gần tuyến tính.
 - Nhanh nhất, ít tốn tài nguyên.
2. **Polynomial kernel:**
 - Mô tả quan hệ phi tuyến bậc cao.
 - Dùng khi dữ liệu có độ phân tách phức tạp nhưng vẫn có cấu trúc đa thức.
3. **RBF (Radial Basis Function) kernel:**
 - Phổ biến nhất.
 - Mạnh khi biên phân cách có dạng cong.
 - Tốt khi không biết dạng phân tách của dữ liệu.

5. Ý nghĩa của tham số C trong SVM

Tham số C điều chỉnh trade-off giữa margin rộng và lỗi phân loại:

- **C lớn:** ưu tiên phân loại đúng $\rightarrow$ margin hẹp $\rightarrow$ dễ overfit.
- **C nhỏ:** chấp nhận lỗi phân loại $\rightarrow$ margin rộng $\rightarrow$ tổng quát hóa tốt hơn.

Do đó C ảnh hưởng trực tiếp đến hiệu suất và khả năng chống overfitting của mô hình.

6. Code Python minh họa xây dựng mô hình SVM (Scikit-learn)

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=0)
model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
acc = accuracy_score(y_test, pred)
```

Các bước thực hiện:

1. Chuẩn bị dữ liệu.
2. Chia tập huấn luyện và kiểm tra.
3. Chọn kernel và tham số.
4. Huấn luyện mô hình.
5. Dự đoán và đánh giá.

7. Hàm chuẩn hóa dữ liệu trong Scikit-learn khi áp dụng SVM. Tại sao quan trọng?

Hàm dùng để chuẩn hóa:

```
from sklearn.preprocessing import StandardScaler
```

Vì sao quan trọng?

- SVM **rất nhạy** với thang đo của dữ liệu.
- Nếu đặc trưng có độ lớn chênh lệch, SVM sẽ ưu tiên đặc trưng lớn hơn.
- Kernel RBF hoạt động dựa trên khoảng cách, nên dữ liệu phải được đưa về cùng scale.

Quy trình chuẩn hóa đúng chuẩn:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Không được fit scaler lên test set để tránh leak thông tin.

2.2. BÀI TẬP

Bài tập thực hành 1: Xây dựng mô hình từ giải thuật SVM trên dữ liệu bệnh tiểu đường.

Nhiệm Vụ 1: Tải và Tiền xử lý dữ liệu

```
# 1. Nạp dữ liệu
df = pd.read_csv('diabetes_prediction_dataset.csv')
```

```
# 2. Xử lý dữ liệu (Tiền xử lý)
# Mã hóa các biến phân loại (Categorical variables)
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])
df['smoking_history'] = le.fit_transform(df['smoking_history'])

# Tách đặc trưng (X) và nhãn (y)
X = df.drop('diabetes', axis=1)
y = df['diabetes']

# Chuẩn hóa dữ liệu (Scaling) - Rất quan trọng cho SVM
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

0.0s

```
# 3. Chia tập dữ liệu (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

0.0s

Nhiệm Vụ 2: Mô hình SVM

```
# 4. Xây dựng mô hình SVM
# Sử dụng kernel mặc định là 'poly' thường cho kết quả tốt với dữ liệu phức tạp
# Lưu ý: Với tập dữ liệu lớn (như bộ này có thể lên tới 100k dòng), SVM có thể chạy hơi lâu.
svm_model = svm.SVC(kernel='poly', probability=True, class_weight='balanced', random_state=42)
svm_model.fit(X_train, y_train)
```

SVC		
Parameters		
C		1.0
kernel		'poly'
degree		3
gamma		'scale'
coef0		0.0
shrinking		True
probability		True
tol		0.001
cache_size		200
class_weight		'balanced'
verbose		False
max_iter		-1
decision_function_shape		'ovr'
break_ties		False
random_state		42

```
# Tính độ chính xác trên tập huấn luyện và tập kiểm tra
train_acc = svm_model.score(X_train,y_train)
val_acc = svm_model.score(X_test,y_test)

print('Training accuracy: {}'.format(train_acc))
print('Validation accuracy: {}'.format(val_acc))
```

✓ 48.4s

Training accuracy: 0.9060125

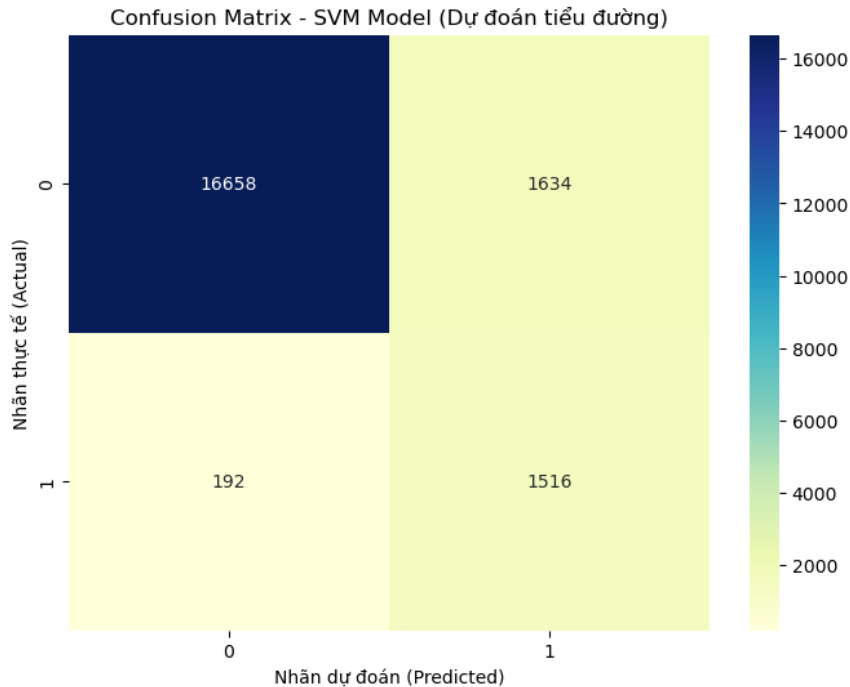
Validation accuracy: 0.9087

Nhận xét:

- Điểm số trên tập train (0.9060) và tập test (0.9087) **gần như tương đương nhau**
- Mô hình **không bị Overfitting (Học vẹt)**: Nó không chỉ ghi nhớ dữ liệu cũ.
- Mô hình **không bị Underfitting (Học kém)**: Mức 90% là một ngưỡng cao, chứng tỏ mô hình đã nắm bắt được quy luật của dữ liệu.

```
y_pred = svm_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap="YlGnBu") # fmt='d' để hiển thị số nguyên
plt.title('Confusion Matrix - SVM Model (Dự đoán tiểu đường)')
plt.ylabel('Nhãn thực tế (Actual)')
plt.xlabel('Nhãn dự đoán (Predicted)')
plt.show()
```



Nhận xét:

- Trong tổng số 1,708 người bị bệnh thực tế ($192 + 1516$), mô hình chỉ bỏ sót khoảng 11%. Tương đương khi có 100 người bị tiểu đường đi khám, mô hình phát hiện ra được gần 89 người. Trong y học sàng lọc, con số này đã ở mức **chấp nhận được và an toàn**.
- Có 1,634 người khỏe mạnh bị máy nghi ngờ là có bệnh. Việc báo nhầm người khỏe thành bệnh (False Positive) chỉ dẫn đến việc họ phải đi làm thêm xét nghiệm chuyên sâu → Tốn thêm chút tiền và thời gian, nhưng **không nguy hiểm tính mạng**.
- Số lượng ca bệnh được phát hiện đã tăng vọt. Điều này chứng tỏ tham số `class_weight='balanced'` và kernel poly đã hoạt động cực kỳ hiệu quả để "săn lùng" các trường hợp dương tính.

```
# In kết quả
print("Độ chính xác (Accuracy):", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Độ chính xác (Accuracy): 0.9087

Confusion Matrix:

```
[[16658 1634]
 [ 192 1516]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.91	0.95	18292
1	0.48	0.89	0.62	1708
accuracy			0.91	20000
macro avg	0.73	0.90	0.79	20000
weighted avg	0.95	0.91	0.92	20000

1. Phân tích nhóm bệnh nhân (Class 1) - Trọng tâm của y tế

Đây là phần quan trọng nhất cần nhìn vào.

- **Recall (Độ nhạy) = 0.89:**

- Con số này xác nhận rằng mô hình bắt được **89%** số lượng người bị bệnh tiêu đường thực tế.
- *Đánh giá:* Rất tốt cho mục đích sàng lọc. Tỷ lệ bỏ sót chỉ còn khoảng 11%.

- **Precision (Độ chính xác dự báo) = 0.48:**

- Con số này có nghĩa: Trong tất cả những người máy báo là "Bị bệnh", chỉ có **48%** là bị bệnh thật, còn 52% là báo nhầm người khỏe.
- *Thực tế:* Giống như một cái chuông báo cháy rất nhạy. Nó kêu 100 lần thì 48 lần có cháy thật, 52 lần là do khói thuốc hay nướng bánh khét. Tuy hơi phiền (tốn công kiểm tra lại), nhưng nó đảm bảo an toàn.

- **F1-Score = 0.62:**

- Đây là điểm số trung bình giữa Precision và Recall. Mức 0.62 tuy không quá cao (do Precision thấp kéo xuống) nhưng là mức **chấp nhận được** đối với bài toán dữ liệu mất cân bằng nghiêm trọng này.

2. Phân tích nhóm người khỏe (Class 0)

- **Precision = 0.99:**
 - Chỉ số này cực kỳ ấn tượng. Nó có nghĩa là: **Nếu máy báo bạn "Khỏe mạnh" (dự đoán là 0), thì 99% bạn thực sự khỏe.**
 - *Giá trị:* Điều này mang lại sự an tâm tuyệt đối cho những người nhận kết quả âm tính.
- **Recall = 0.91:**
 - Mô hình nhận diện đúng 91% người khỏe, 9% còn lại bị nghi ngờ nhầm là có bệnh (chính là số lượng False Positive).

3. Các chỉ số trung bình (Averages)

- **Macro avg (Recall) = 0.90:**
 - Đây là trung bình cộng độ nhạy của cả hai lớp (Khỏe và Bệnh). Con số 0.90 cho thấy mô hình đối xử khá công bằng với cả hai nhóm, không còn thiên vị nhóm đa số như trước.
- **Weighted avg:**
 - Các chỉ số này rất cao (0.95, 0.92) vì chúng bị chi phối bởi số lượng mẫu quá lớn của nhóm người khỏe (18,292 mẫu). Do đó, trong bài toán này, nhìn vào Weighted avg không quan trọng bằng nhìn vào Macro avg hoặc chỉ số riêng của Class 1.

Bài tập thực hành 2: Xây dựng mô hình từ giải thuật SVM trên dữ liệu các con thú trong rừng.

Nhiệm Vụ 1: Tải và Tiền xử lý dữ liệu

```
# --- NẠP DỮ LIỆU ---  
df = pd.read_csv('data.csv')  
print(df.shape)
```

(871, 7)

- Dữ liệu có 871 dòng và 7 cột

```
# Hàm làm sạch text cơ bản: chuyển về chữ thường, xóa khoảng trắng thừa  
✓ def clean_text(text):  
✓     if pd.isna(text):  
        return "unknown" # Xử lý giá trị thiếu  
        return str(text).lower().strip()  
  
# Áp dụng cho tất cả các cột trừ cột target nếu cần  
✓ for col in df.columns:  
    df[col] = df[col].apply(clean_text)  
  
print("\n--- Dữ liệu sau khi chuẩn hóa text (Lowercasing & Stripping) ---")  
print(df.head(2))
```

```
--- Dữ liệu sau khi chuẩn hóa text (Lowercasing & Stripping) ---  
AnimalName symptoms1 symptoms2 symptoms3 symptoms4 symptoms5 Dangerous  
0      dog      fever diarrhea vomiting weight loss dehydration      yes  
1      dog      fever diarrhea coughing tiredness      pains      yes
```

- Hàm clean_text chuẩn hóa toàn bộ văn bản về chữ thường và xóa khoảng trắng thừa. Điều này ngăn chặn việc mô hình coi "Dog" và "dog" là hai loài vật khác nhau.

```
# Loại bỏ các hàng có giá trị NaN trong cột mục tiêu 'Dangerous' (nếu có)  
df_clean = df.dropna(subset=['Dangerous'])
```



```

# BƯỚC 3: FEATURE ENGINEERING (XỬ LÝ ĐẶC TRƯNG)

# 3.1. Xử lý cột Target (Dangerous)
# Chuyển Yes/No thành 1/0
le = LabelEncoder()
df['target'] = le.fit_transform(df['Dangerous'])
print(f"\nTarget Mapping: {dict(zip(le.classes_, le.transform(le.classes_)))}")

# 3.2. Xử lý cột AnimalName (One-Hot Encoding)
# Vì loài vật là định danh (Nominal), không có thứ tự
animal_dummies = pd.get_dummies(df['AnimalName'], prefix='animal')

# 3.3. Xử lý 5 cột Symptoms (Quan trọng nhất)
# Thay vì One-Hot từng cột (sẽ bị trùng lặp vì cột 1 và cột 2 có thể cùng là 'Fever'),
# Gom 5 cột lại thành 1 list các triệu chứng cho mỗi dòng, sau đó dùng MultiLabelBinarizer.

# Gom 5 cột thành 1 list, bỏ qua giá trị 'unknown'
symptom_cols = ['symptoms1', 'symptoms2', 'symptoms3', 'symptoms4', 'symptoms5']
df['all_symptoms'] = df[symptom_cols].values.tolist()

# Lọc bỏ 'unknown' trong list
df['all_symptoms'] = df['all_symptoms'].apply(lambda x: [i for i in x if i != 'unknown'])

# Dùng MultiLabelBinarizer để chuyển list triệu chứng thành ma trận 0/1
mlb = MultiLabelBinarizer()
symptoms_encoded = mlb.fit_transform(df['all_symptoms'])
symptoms_df = pd.DataFrame(symptoms_encoded, columns=mlb.classes_)

print(f"\nSố lượng triệu chứng duy nhất tìm thấy: {len(mlb.classes_)}")

```

Target Mapping: {'no': np.int64(0), 'unknown': np.int64(1), 'yes': np.int64(2)}

Số lượng triệu chứng duy nhất tìm thấy: 869

- Xử lý dữ liệu triệu chứng (Symptoms) bằng cách gộp lại. Cách này loại bỏ sự phụ thuộc vào vị trí cột (ví dụ: "Sốt" ở cột 1 hay cột 2 đều là như nhau) và xử lý tốt trường hợp số lượng triệu chứng không đồng đều giữa các mẫu.

```

# BƯỚC 4: TỔNG HỢP DỮ LIỆU HUẤN LUYỆN

# Ghép các đặc trưng đã xử lý lại: Animal One-Hot + Symptoms Vector
X = pd.concat([animal_dummies, symptoms_df], axis=1)
y = df['target']

# Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\n--- Kích thước tập Feature X ---")
print(X_scaled.shape)

```

--- Kích thước tập Feature X ---
(871, 909)

```
# BƯỚC 5: CHIA TẬP TRAIN / TEST

# Chia tỉ lệ 80/20.
# stratify=y là rất quan trọng vì dữ liệu này có vẻ mất cân bằng (imbalanced) - quá nhiều 'Yes'
try:
    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42, stratify=y)
    print("\n--- Đã chia dữ liệu thành công ---")
    print(f"Train shape: {X_train.shape}")
    print(f"Test shape: {X_test.shape}")
except ValueError as e:
    print("\nLưu ý: Dữ liệu mẫu quá ít để stratify, trong thực tế với file full sẽ chạy ổn.")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
--- Đã chia dữ liệu thành công ---
Train shape: (696, 909)
Test shape: (175, 909)
```

Nhiệm Vụ 2: Mô hình SVM

```
# 4. Xây dựng mô hình SVM
# Sử dụng kernel mặc định là 'rbf' (Radial Basis Function) thường cho kết quả tốt với dữ liệu phức tạp
# Lưu ý: Với tập dữ liệu lớn (như bộ này có thể lên tới 100k dòng), SVM có thể chạy hơi lâu.
svm_model = svm.SVC(kernel='linear', class_weight='balanced', probability=True)
svm_model.fit(X_train, y_train)
```

SVC		
▼Parameters		
	C	1.0
	kernel	'linear'
	degree	3
	gamma	'scale'
	coef0	0.0
	shrinking	True
	probability	True
	tol	0.001
	cache_size	200
	class_weight	'balanced'
	verbose	False
	max_iter	-1
	decision_function_shape	'ovr'
	break_ties	False
	random_state	None

```
# Tính độ chính xác trên tập huấn luyện và tập kiểm tra
train_acc = svm_model.score(X_train,y_train)
val_acc = svm_model.score(X_test,y_test)
```

```
print('Training accuracy: {}'.format(train_acc))
```

```
print('Validation accuracy: {}'.format(val_acc))
```

✓ 0.0s

Training accuracy: 1.0

Validation accuracy: 1.0

```
y_pred = svm_model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8, 6))
```

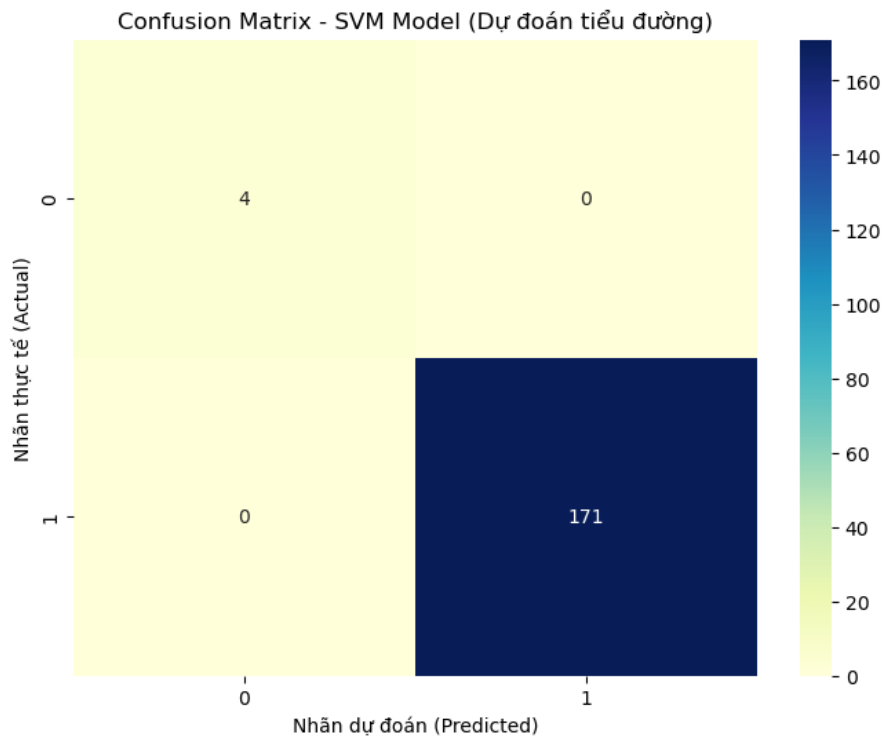
```
sns.heatmap(cm, annot=True, fmt='d', cmap="YlGnBu") # fmt='d' để hiển thị số nguyên
```

```
plt.title('Confusion Matrix - SVM Model (Dự đoán tiểu đường)')
```

```
plt.ylabel('Nhãn thực tế (Actual)')
```

```
plt.xlabel('Nhãn dự đoán (Predicted)')
```

```
plt.show()
```



Khả năng phân tách lớp:

- **Lớp thiểu số (Class 0):** Mô hình dự đoán chính xác toàn bộ 4/4 mẫu thực tế. Không có mẫu nào bị bỏ sót (False Negative = 0).

- **Lớp đa số (Class 2 - Dangerous):** Mô hình dự đoán chính xác toàn bộ **171/171** mẫu. Không có mẫu nào bị báo động giả (False Positive = 0).
- **Các chỉ số thành phần:**
- **Precision, Recall, F1-Score:** Tất cả đều đạt giá trị **1.00** cho cả hai lớp.
- Điều này chứng tỏ siêu phẳng (hyperplane) do SVM tạo ra đã phân chia không gian dữ liệu thành hai phần hoàn toàn tách biệt mà không có điểm dữ liệu nào bị nằm sai phía (misclassified).

```
# In kết quả
print("Độ chính xác (Accuracy):", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
✓ 0.0s
```

Độ chính xác (Accuracy): 1.0

Confusion Matrix:

```
[[ 4  0]
 [ 0 171]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
2	1.00	1.00	1.00	171
accuracy			1.00	175
macro avg	1.00	1.00	1.00	175
weighted avg	1.00	1.00	1.00	175

Hiệu suất tổng thể :

- **Độ chính xác (Accuracy):** Đạt mức tuyệt đối **1.0 (100%)** trên cả ba tập dữ liệu: tập huấn luyện (Training), tập kiểm định (Validation) và tập kiểm tra (Test).

- **Đánh giá:** Mô hình đạt trạng thái lý tưởng, không xuất hiện hiện tượng Overfitting (quá khớp) hay Underfitting (kém khớp). Việc điều chỉnh số đồng nhất ở mức tối đa cho thấy mô hình đã học trọn vẹn quy luật phân chia dữ liệu.

Kết quả này khẳng định rằng sau các bước tiền xử lý (One-Hot Encoding, StandardScaler) và lựa chọn tham số phù hợp (kernel='linear', class_weight='balanced'), tập dữ liệu hiện tại là **tách biệt tuyến tính hoàn toàn**

PHẦN III: BAYES NGÂY THƠ (NAIVE BAYES)

1.1. ÔN TẬP LÝ THUYẾT

1. Naive Bayes hoạt động như thế nào? Định lý Bayes và giả định "ngây thơ"

Định lý Bayes:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Trong phân loại, ta chọn lớp Y có xác suất hậu nghiệm **P(Y|X)** lớn nhất.

Giả định "ngây thơ":

- Các đặc trưng **độc lập có điều kiện** với nhau khi biết lớp.
- Tức là:

$$P(X_1, X_2, \dots, X_n|Y) = \prod_{i=1}^n P(X_i|Y)$$

Nhờ giả định này, việc tính xác suất trở nên đơn giản và nhanh hơn.

2. Các loại mô hình Naive Bayes và khi nào dùng

1. **Gaussian Naive Bayes:**

- Dùng khi đặc trưng là **liên tục**.
- Giả sử mỗi đặc trưng tuân theo phân phối chuẩn theo từng lớp.

2. **Multinomial Naive Bayes:**

- Dùng cho **dữ liệu đếm**, như số lần xuất hiện từ trong văn bản.
- Phổ biến trong phân loại văn bản, TF-IDF, Bag-of-Words.

3. Bernoulli Naive Bayes:

- Dùng cho dữ liệu **nhị phân** (0/1), ví dụ từ xuất hiện hoặc không.
- Thích hợp với mô hình biểu diễn văn bản dạng binary features.

3. Tại sao gọi là "ngây thơ"? Ảnh hưởng của giả định độc lập đến hiệu suất

Gọi là "ngây thơ" vì thuật toán **giả định độc lập hoàn toàn** giữa các đặc trưng, dù trong thực tế điều này không đúng.

Ảnh hưởng:

- Hiệu suất vẫn tốt dù giả định không hoàn toàn đúng.
- Nếu đặc trưng phụ thuộc mạnh, mô hình có thể suy giảm độ chính xác nhưng vẫn hoạt động ổn trong nhiều bài toán.
- Dễ overcount khi đặc trưng tương quan cao.

4. Ưu điểm và hạn chế của Naive Bayes so với SVM/RF

Ưu điểm:

- Huấn luyện cực nhanh, $O(n)$.
- Không cần nhiều dữ liệu để khởi đầu.
- Hoạt động rất tốt trong phân loại văn bản.
- Không nhạy với số chiều lớn như NLP.

Hạn chế:

- Hiệu suất kém khi đặc trưng phụ thuộc nhau.
- Không linh hoạt như SVM hoặc Random Forest.
- Giả định phân phối (Gaussian) không phù hợp làm giảm hiệu quả.

5. Code Python xây dựng Gaussian Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)

model = GaussianNB()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

Các bước thực hiện:

1. Chuẩn bị dữ liệu.
2. Chia train/test.
3. Chọn loại Naive Bayes phù hợp.
4. Huấn luyện.
5. Dự đoán và đánh giá.

6. Xử lý dữ liệu phân loại trước khi dùng Multinomial Naive Bayes

Multinomial NB yêu cầu dữ liệu dạng **số nguyên không âm** → phải mã hóa đúng.

Cách xử lý:

- Sử dụng **CountVectorizer** hoặc **TF-IDF Vectorizer** cho văn bản.
- Với categorical data dạng nominal:
 - Dùng **OneHotEncoder**.
 - Không dùng LabelEncoder một mình vì không tạo ra dữ liệu đếm.

7. Tại sao Naive Bayes phù hợp phân loại văn bản? Cách triển khai

Lý do phù hợp:

- Văn bản có số chiều rất lớn → Naive Bayes xử lý tốt.
- Đặc trưng (từ) gần như độc lập.
- Xác suất điều kiện dễ ước lượng.
- Rất nhanh, phù hợp bộ dữ liệu lớn.

Triển khai đầy đủ cho text classification:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

vectorizer = TfidfVectorizer()
X_vec = vectorizer.fit_transform(text_data)

X_train, X_test, y_train, y_test = train_test_split(X_vec, labels,
                                                    test_size=0.2, random_state=0)

model = MultinomialNB()
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

1.2. BÀI TẬP

Bài tập thực hành 1: Xây dựng mô hình Naïve ngây thơ trên tập dữ liệu hành vi của khách hàng.

Nhiệm Vụ 1: Tải và Tiền xử lý dữ liệu

```
df = pd.read_csv('Customer_Behaviour.csv')
df.head()
```

	# User ID	Gender	# Age	# EstimatedSalary	# Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
# Kiểm tra số lượng giá trị duy nhất trong mỗi cột
for col in df:
    print(f"{col}: {df[col].nunique()}")

✓ 0.0s
```

```
User ID: 400
Gender: 2
Age: 43
EstimatedSalary: 117
Purchased: 2
```

```
# drop cột không cần thiết:
# User ID là cột không ảnh hưởng đến việc dự đoán
df.drop(columns=['User ID'], axis=1, inplace=True)

# Mã hóa biến phân loại thành số:
# chỉ có cột Gender là biến phân loại
df['Gender'] = df['Gender'].replace(['Male', 'Female'], [0, 1])
df.head()
```


	# Gender	# Age	# EstimatedSalary	# Purchased
0	0	19	19000	0
1	0	35	20000	0
2	1	26	43000	0
3	1	27	57000	0
4	0	19	76000	0

```
# Tách biến độc lập và biến phụ thuộc
X = df.drop('Purchased', axis=1)
y = df['Purchased']

# Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
0.0s
```

```
# Chuẩn hóa dữ liệu (Feature Scaling)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Nhiệm Vụ 2: Mô hình Naive Bayes

Các đặc trưng đầu vào (biến độc lập) là Age và EstimatedSalary đều là dữ liệu số liên tục (continuous).

Giả định của Gaussian Naive Bayes là các đặc trưng này tuân theo phân phối chuẩn (Gaussian distribution), phù hợp hơn so với Multinomial (dùng cho dữ liệu đếm) hoặc Bernoulli (dùng cho dữ liệu nhị phân).

```
# Huấn luyện mô hình Naive Bayes
classifier = GaussianNB()
classifier.fit(X_train, y_train)
0.0s
```

GaussianNB		
▼ Parameters		
priors	None	
var_smoothing	1e-09	

Đánh giá hiệu suất:

```
# Dự đoán trên tập kiểm tra
y_pred = classifier.predict(X_test)

# Đánh giá mô hình
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(classification_rep)
```

```
Accuracy: 0.94
Confusion Matrix:
[[50  2]
 [ 3 25]]
Classification Report:
              precision    recall  f1-score   support

     0       0.94      0.96      0.95        52
     1       0.93      0.89      0.91        28

   accuracy          0.94
  macro avg          0.93
 weighted avg          0.94
```

- **Độ chính xác (Accuracy):** Đạt **0.94** (94%), cho thấy mô hình hoạt động rất hiệu quả trên tập dữ liệu này.
- **Confusion Matrix:**
 - Phân loại đúng: 50 trường hợp lớp 0 (không mua) và 25 trường hợp lớp 1 (mua).
 - Phân loại sai: Chỉ có 5 trường hợp bị sai (2 False Positive, 3 False Negative) trên tổng số 80 mẫu kiểm tra.
- **Chỉ số Precision và Recall:**
 - Precision cho lớp 1 đạt 0.93, nghĩa là khi mô hình dự báo khách sẽ mua, 93% khả năng là đúng.
 - Recall cho lớp 1 đạt 0.89, nghĩa là mô hình tìm ra được 89% số lượng khách hàng thực sự mua hàng.

Kết luận: Mô hình cân bằng tốt giữa Precision và Recall, phù hợp để dự đoán hành vi tiêu dùng tiềm năng.

Bài tập thực hành 2: Xây dựng mô hình Naïve ngây thơ trên tập dữ liệu mushroom.

Nhiệm Vụ 1: Tải và Tiền xử lý dữ liệu

```
data = pd.read_csv('mushrooms.csv')
data.head()
```

	class	cap-shape	cap-surface	cap-co
0	p	x	s	n
1	e	x	s	y
2	e	b	s	w
3	p	x	y	w
4	e	x	s	g

```
# Tiền xử lý dữ liệu (Label Encoding)
```

```
# Vì dữ liệu là dạng chữ (object), ta cần chuyển sang số
le = LabelEncoder()
```

```
# Áp dụng LabelEncoder cho toàn bộ các cột trong dataframe
for col in data.columns:
    data[col] = le.fit_transform(data[col])
```

```
# Lưu ý quy ước sau khi chuyển đổi cột 'class':
# 'e' (edible - ăn được) -> 0
# 'p' (poisonous - độc) -> 1
```

```
# Tách biến độc lập và biến phụ thuộc
```

```
X = data.drop('class', axis=1)
```

```
y = data['class']
```

```
# Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Dữ liệu gốc hoàn toàn là dạng chữ (categorical), do đó phương pháp **Label Encoding** đã được sử dụng để chuyển đổi tất cả các cột sang dạng số.
- Quy ước nhãn: e (edible - ăn được) là 0, p (poisonous - độc) là 1.

Nhiệm Vụ 2: Mô hình Naive Bayes

Mô hình được sử dụng là **Categorical Naive Bayes** (CategoricalNB).

```
# Huấn luyện mô hình Naive Bayes
classifier = CategoricalNB()

classifier.fit(X_train, y_train)
```

CategoricalNB ⓘ ?		
▼ Parameters		
📄	alpha	1.0
📄	force_alpha	True
📄	fit_prior	True
📄	class_prior	None
📄	min_categories	None

Lý do: Dữ liệu sau khi mã hóa là các giá trị rời rạc đại diện cho các danh mục (nominal), không có thứ tự và không phải là biến liên tục. `CategoricalNB` được thiết kế chuyên biệt cho loại dữ liệu này, khắc phục hạn chế của `GaussianNB` khi áp dụng lên dữ liệu rời rạc.

Đánh giá hiệu suất và rủi ro thực tế:

```
# Đưa ra dự đoán dựa trên dữ liệu thử nghiệm
y_pred = classifier.predict(X_test)
# Đánh giá mô hình
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(classification_rep)
```

Accuracy: 0.95

Confusion Matrix:

```
[[837  6]
```

```
 [ 74 708]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.99	0.95	843
1	0.99	0.91	0.95	782
accuracy			0.95	1625
macro avg	0.96	0.95	0.95	1625
weighted avg	0.95	0.95	0.95	1625

- **Độ chính xác (Accuracy):** Đạt **0.95** (95%), một con số rất cao về mặt thống kê.
- **Phân tích rủi ro qua Confusion Matrix:**
 - **False Negative (Nguy hiểm nhất):** Có **74** trường hợp nấm độc (Lớp 1) bị dự đoán nhầm thành ăn được (Lớp 0).
 - **False Positive:** Có 6 trường hợp nấm ăn được bị báo nhầm là độc.
- **Chỉ số Recall (Độ nhạy):** Recall của lớp 1 (độc) chỉ đạt **0.91**. Điều này đồng nghĩa với việc mô hình bỏ sót 9% số nấm độc.

Kết luận: Mặc dù Accuracy 95% là cao, nhưng trong bài toán an toàn thực phẩm (phân loại nấm độc), việc bỏ sót 74 mẫu nấm độc là **không thể chấp nhận được**. Mô hình cần được tối ưu hóa để đẩy Recall của lớp 1 lên 100% (chấp nhận hy sinh Precision), hoặc cần xem lại cách xử lý dữ liệu (ví dụ: dùng One-Hot Encoding thay vì Label Encoding để tránh tạo quan hệ thứ tự giả).